

講義 20：Control Break 群組小計（授課順序：接在講義 11 之後）

對應練習：[ex20](#) | 答案程式：[ZR_TR20_CONTROL_BREAK](#)

本講重點

- 群組報表的標準需求：明細 → 各組小計 → 總計
- Control Break 陳述式：[AT NEW](#) / [AT END OF](#) / [AT FIRST](#) / [AT LAST](#)
- [SUM](#)：把整組數值欄自動加總進 work area
- 兩條鐵則：先 **[SORT](#)**、**群組欄位放結構前面**
- [AT](#) 區塊內欄位被遮蔽成 * 的規則

1. 需求長相

「航班營收報表，依航空公司小計，最後一行總計」——傳統紙本報表的招牌格式：

```
=== 航班營收（依公司小計） ===
AA American Airlines
  0017 2026/01/03      380      159,494.31
  0017 2026/01/31      372      156,136.13
  小計                  752      315,630.44
AZ Alitalia
  ...
  小計                  ...
=====
總計                    5,204  2,381,904.99
```

土法煉鋼要自己記「上一筆的 carrid」逐筆比對；ABAP 直接內建了這個場景的語法——Control Break。

2. 語法：LOOP 裡的 AT 區塊

```
SORT gt_rev BY carrid connid fldate.           " 鐵則一：先依群組欄位排序！

LOOP AT gt_rev INTO gs_rev.

  AT FIRST.                                     " 整個 LOOP 的第一筆之前
    WRITE / '=== 航班營收（依公司小計） ==='.
  ENDAT.

  AT NEW carrid.                                " 每組的第一筆之前：組頭
    WRITE: / gs_rev-carrid, gs_rev-carrname.
  ENDAT.
```

```

WRITE: /5 gs_rev-connid, gs_rev-fldate,      " 明細行 ( 一般輸出 )
        gs_rev-seatsocc, gs_rev-revenue.

AT END OF carrid.                          " 每組的最後一筆之後：小計
SUM.                                         " 數值欄加總進 gs_rev
WRITE: /5 '小計', gs_rev-seatsocc, gs_rev-revenue.
ULINE.
ENDAT.

AT LAST.                                    " 整個 LOOP 的最後一筆之後：總計
SUM.
WRITE: / '總計', gs_rev-seatsocc, gs_rev-revenue.
ENDAT.

ENDLOOP.

```

陳述式	觸發時機	典型用途
AT FIRST ... ENDAT	迴圈第一筆之前，執行一次	報表標題
AT NEW 欄位	該欄位 (含其左邊所有欄位) 值改變的那一筆之前	組頭
AT END OF 欄位	該組最後一筆之後	小計
AT LAST ... ENDAT	迴圈最後一筆之後，執行一次	總計

3. SUM：整組自動加總

SUM 只能寫在 AT ... ENDAT 區塊內：把目前群組 (AT FIRST/AT LAST 則是全部) 所有列的數值型欄位 (i、p、f) 加總，結果放進 work area 的對應欄位。所以小計行直接 WRITE gs_rev-revenue 就是組合計——不用自己宣告累加變數。

注意：SUM 是「所有數值欄」一起總——結構裡若有不該加總的數值欄 (如單價)，小計行不要去印它 (值是無意義的總和)。

4. 兩條鐵則與遮蔽規則

鐵則一：先 SORT

Control Break 只認「相鄰列」：沒依群組欄位排序，同公司資料分散各處，AT NEW/AT END OF 會觸發好幾次，小計整組錯——而且不報錯。SORT 的欄位順序要跟群組層次一致。

鐵則二：群組欄位放結構最前面

AT NEW f 的比較規則是「f 加上它左邊所有欄位」有任一變動就觸發。所以結構欄位順序要照「群組層次」排：

```

TYPES: BEGIN OF ty_rev,
        carrid    TYPE sflight-carrid,      " 群組欄位放最前

```

```

carrname TYPE scarr-carrname,      " 跟 carrid 一對一，緊跟其後
connid   TYPE sflight-connid,      " 明細鍵
fldate   TYPE sflight-fldate,
seatsocc TYPE sflight-seatsocc,     " 數值欄 ( 要小計的 )
revenue  TYPE p LENGTH 12 DECIMALS 2,
END OF ty_rev.

```

把 **carrid** 放在中間，前面的欄位一變就誤觸發——群組欄位排最前是保命慣例。

遮蔽規則：AT 區塊內右邊欄位變 *

進入 **AT NEW f / AT END OF f** 區塊時，work area 中 **f** 右邊的字元類欄位全部被遮成 *、數值欄清為初始值（明細值沒有意義了，系統防止誤用）。上例組頭想同時印 **carrid** 與 **carrname**，就要用 **AT NEW carrname**（它與 **carrid** 一對一，且依規則「**carrname** 及其左邊的 **carrid**」任一變動才觸發，效果等同 **AT NEW carrid**，但兩個欄位都看得到）：

```

AT NEW carrname.                  " 用右邊那個欄位當斷點，組頭才印得到兩欄
    WRITE: / gs_rev-carrid, gs_rev-carrname.
ENDAT.

```

第一次看到 ********* 出現在報表上，幾乎都是這個遮蔽規則造成的。

5. 適用邊界

- Control Break 專屬 **LOOP AT ... INTO** (work area 型)；搭配 **ASSIGNING** 不能用 AT 區塊。
- 迴圈若加了 **WHERE** 條件或中途 **DELETE**，群組判斷可能失真——要過濾就先把資料整理成乾淨的內表再 LOOP。
- 只要「總計」不要明細時，別用 **LOOP+AT LAST**，直接 **SELECT SUM(...) GROUP BY** (資料庫端彙總) 或 **COLLECT** 更省——課程期末後可自行延伸。

6. 常見錯誤與陷阱

症狀	原因
同一公司出現多次小計	沒先 SORT (或 SORT 欄位跟 AT 欄位不一致)
組頭/小計行印出 *****	遮蔽規則：AT 區塊內右邊字元欄被遮——改用右側欄位當 AT 斷點或別印
小計數字大得離譜	SUM 加總了不該總的數值欄 (如單價) 還把它印出來
AT NEW 太常觸發	群組欄位左邊還有會變動的欄位——群組欄位移到結構最前
ASSIGNING 迴圈裡 AT 區塊報錯	Control Break 只支援 INTO 形式

7. 課堂練習

完成 [ex20](#)：SFLIGHT + SCARR JOIN 出營收明細，依公司做組頭、小計，AT LAST 總計，並實驗「不 SORT 會怎樣」與遮蔽規則。